Operating system

Topic: **Disk Scheduling**

What is Disk Scheduling Algorithm?

A Process makes the I/O requests to the operating system to access the disk. Disk Scheduling Algorithm manages those requests and decides the order of the disk access given to the requests.

Why is Disk Scheduling Algorithm needed?

Disk Scheduling Algorithms are needed because a process can make multiple I/O requests and multiple processes run at the same time. The requests made by a process may be located at different sectors on different tracks. Due to this, the seek time may increase more. These algorithms help in minimizing the seek time by ordering the requests made by the processes.

Important Terms related to Disk Scheduling Algorithms

Seek Time - It is the time taken by the disk arm to locate the desired track.

Rotational Latency - The time taken by a desired sector of the disk to rotate itself to the position where it can access the Read/Write heads is called Rotational Latency.

Transfer Time - It is the time taken to transfer the data requested by the processes.

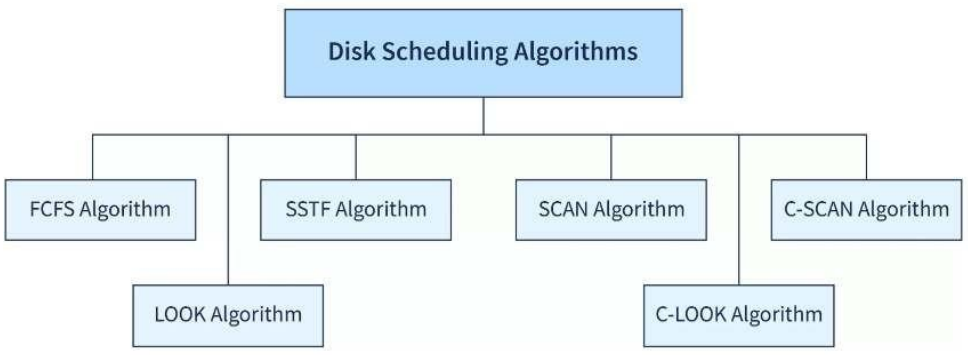
Disk Access Time - Disk Access time is the sum of the Seek Time, Rotational Latency, and Transfer Time.

Disk Response Time: The disk processes one request at a single time. So, the other requests wait in a queue to finish the ongoing process of request. The average of this waiting time is called "Disk Response Time".

Starvation: Starvation is defined as the situation in which a low-priority job keeps waiting for a long time to be executed. The system keeps sending high-priority jobs to the disk scheduler to execute first.

Types of Disk Scheduling Algorithm in OS

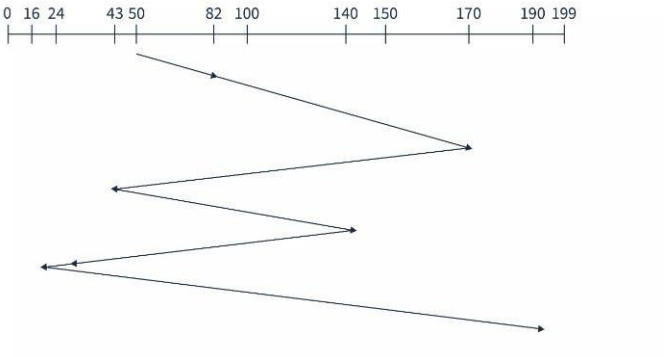
We have various types of Disk Scheduling Algorithms available in our system. Each one has its own capabilities and weak points.



1. FCFS disk scheduling algorithm

It stands for 'first-come-first-serve'. As the name suggests, the request which comes first will be processed first and so on. The requests coming to the disk are arranged in a proper sequence as they arrive. Since every request is processed in this algorithm, so there is no chance of 'starvation'.

Example: Suppose a disk has 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



FCFS disk scheduling algorithm

Explanation: In the above image, we can see the head starts at position 50 and moves to request 82. After serving them the disk arm moves towards the second request, that is 170 and then to the request 43 and so on. In this algorithm, the disk arm will serve the requests in arriving order. In this way, all the requests are served in arriving order until the process executes.

"Seek time" will be calculated by adding the head movement differences of all the

requests: Seek time= "(82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) +

(190-16) = 642

Advantages:

- Implementation is easy.
- No chance of starvation.
- Seek time increases. Not so efficient.

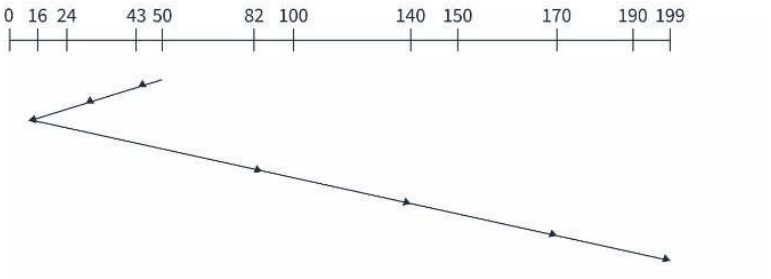
Disadvantages:

- Seek time increases. Not so efficient.

2. SSTF disk scheduling algorithm-

It stands for 'Shortest seek time first'. As the name suggests, it searches for the request having the least 'seek time' and executes them first. This algorithm has less 'seek time' as compared to FCFS Algorithm.

Example: Suppose a disk has 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



Explanation: The disk arm searches for the request which will have the least difference in head movement. So, the least difference is (50-43). Here the difference is not about the shortest value but it is about the shortest time the head will take to reach the nearest next request. So, after 43, the head will be nearest to 24, and from here the head will be nearest to the request 16, After 16, the nearest request is 82, so the disk arm will move to serve request 82 and so on.

Hence, Calculation of Seek Time = (50-43) + (43-24) + (24-16) + (82-16) + (140-82) + (170-140) + (190-170) = 208

Advantages:

In this algorithm, disk response time is

less. More efficient than FCFS.

Disadvantages:

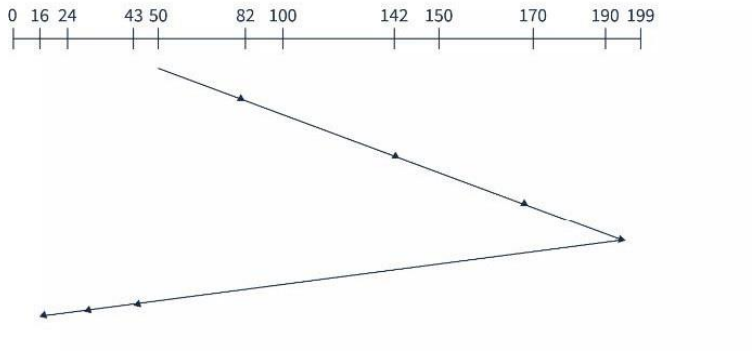
Less speed of algorithm execution.

Starvation can be seen.

3. SCAN disk scheduling algorithm:

In this algorithm, the head starts to scan all the requests in a direction and reaches the end of the disk. After that, it reverses its direction and starts to scan again the requests in its path and serves them. Due to this feature, this algorithm is also known as the "Elevator Algorithm".

Example: Suppose a disk has 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50. The 'disk arm' will first move to the larger values.



Explanation: In the above image, we can see that the disk arm starts from position 50 and goes in a single direction until it reaches the end of the disk i.e.- request position 199. After that, it reverses and starts servicing in the opposite direction until reaches the other end of the disk. This process keeps going on until the process is executed. Hence, Calculation of 'Seek Time' will be like: (199-50) + (199- 16) =332

Advantages:

Implementation is easy.

Requests do not have to wait in a queue.

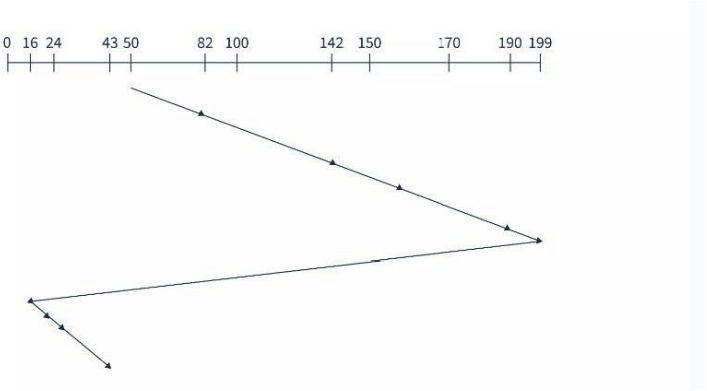
Disadvantage:

The head keeps going on to the end even if there are no requests in that direction.

4. C-SCAN disk scheduling algorithm:

It stands for "Circular-Scan". This algorithm is almost the same as the Scan disk algorithm but one thing that makes it different is that 'after reaching the one end and reversing the head direction, it starts to come back. The disk arm moves toward the end of the disk and serves the requests coming into its path. After reaching the end of the disk it reverses its direction and again starts to move to the other end of the disk but while going back it does not serve any requests.

Example: Suppose a disk has 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



Explanation: In the above figure, the disk arm starts from position 50 and reaches the end (199), and serves all the requests in the path. Then it reverses the direction and moves to the other end of the disk i.e.- 0 without serving any task in the path. After reaching 0, it will again move towards the largest remaining value which is 43. So, the head will start from 0 and moves to request 43 serving all the requests coming in the path. And this process keeps going.

Hence, Seek Time will be = (199-50) + (199-0) + (43-0) =391

Advantages:

The waiting time is uniformly distributed among the requests.

Response time is good in it.

Disadvantages:

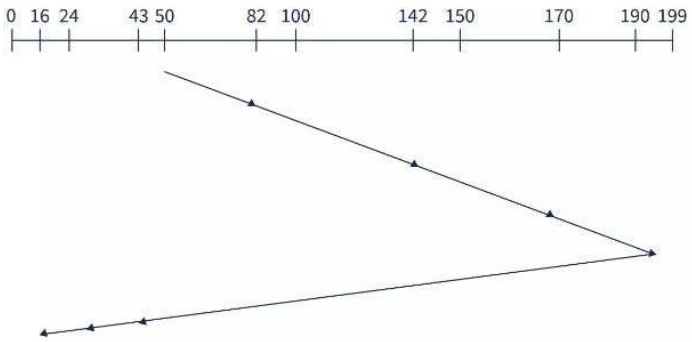
The time taken by the disk arm to locate a spot is increased

here. The head keeps going to the end of the disk.

5. LOOK the disk scheduling algorithm:

In this algorithm, the disk arm moves to the 'last request' present and services them. After reaching the last requests, it reverses its direction and again comes back to the starting point. It does not go to the end of the disk, in spite, it goes to the end of requests.

Example a disk having 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



Explanation: The disk arm starts from 50 and starts to serve requests in one direction only but in spite of going to the end of the disk, it goes to the end of requests i.e.-190. Then comes back to the last request of other ends of the disk and serves them. And again, starts from here and serves till the last request of the first side. Hence, Seek time = (190-50) + (190-16) =314

Advantages:

Starvation does not occur.

Since the head does not go to the end of the disk, the time is not wasted here.

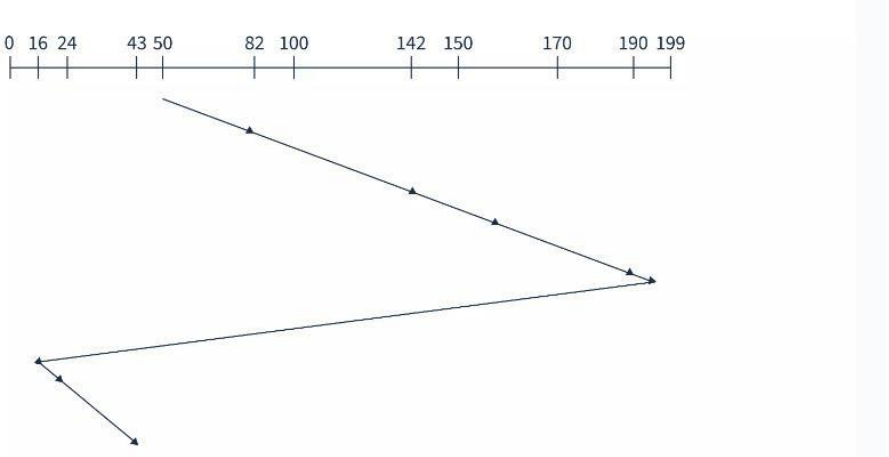
Disadvantage:

The arm has to be conscious to find the last request.

6. C-LOOK disk scheduling algorithm:

The C-Look algorithm is almost the same as the Look algorithm. The only difference is that after reaching the end requests, it reverses the direction of the head and starts moving to the initial position. But in moving back, it does not serve any requests.

Example: Suppose a disk has 200 tracks (0-199). The request sequence (82,170,43,140,24,16,190) are shown in the given figure and the head position is at 50.



Explanation: The disk arm starts from 50 and starts to serve requests in one direction only but in spite of going to the end of the disk, it goes to the end of requests i.e.-190. Then comes back to the last request of other ends of a disk without serving them. And again, starts from the other end of the disk and serves requests of its path. Hence, Seek Time = (190-50) + (190-16) + (43-16) =341

Advantages:

The waiting time is decreased.

If there are no requests till the end, it reverses the head direction immediately.

Starvation does not occur.

The time taken by the disk arm to find the desired spot is less.

Disadvantage:

The arm has to be conscious about finding the last request.

Conclusion:

The Disk Scheduling Algorithm in OS is used to manage input and output requests to the disk.

Disk Scheduling is important as multiple requests are coming to disk simultaneously and it is also known as the "Request Scheduling Algorithm"

Various types of scheduling algorithms are:

FCFS (First come first serve)

algorithm SSTF (Shortest seek time

first) algorithm Scan Disk Algorithm

C-Scan (Circular scan) algorithm

Look algorithm

C-Look (Circular look) algorithm

An algorithm in which starvation is minimum is considered an efficient algorithm. The given figure and the head start is at request 50.